

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one alone.

2026-09-19

Rules, judge, loop

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one alone.

Timing: check-in 5 · what an agent is + tool calls + ReAct/harness + chronology 10 · rulebook + live AGENTS.md 10 · judge 5 · loop 5 · feature 2 kick-off live 13 · your turn 32 · retro 7 · wrap 3 = 90. TA-run. The slides carry the ideas; every command they need is on the Week 4 page, open on their laptops. Presenting laptop: SplitIt repo with bugs 000/001 fixed and feature 1 merged, on main, clean; Codex open on it; GitHub Desktop signed in; data reset.

What we'll do.

- 1 Check-in — feature 1, merged.
- 2 What an agent actually is — and how it got that way.
- 3 Three standards — a rulebook, a judge, a loop.
- 4 Feature 2, live — we watch it build, then we check its work.
- 5 Your turn — one bug, one feature. Thirty-two minutes.
- 6 Retro — turn today's worst moment into one rule.

2026-09-19

Rules, judge, loop

└─What we'll do.

CLASS 4 · TODAY

What we'll do.

- 1 **Check-in** — feature 1, merged.
- 2 **What an agent actually is** — and how it got that way.
- 3 **Three standards** — a rulebook, a judge, a loop.
- 4 **Feature 2, live** — we watch it build, then we check its work.
- 5 **Your turn** — one bug, one feature. Thirty-two minutes.
- 6 **Retro** — turn today's worst moment into one rule.

Show me your GitHub page.

- A branch for feature 1.
- A merged pull request.
- Not there yet? Do it first today. Everything else still works.

2026-09-19

Rules, judge, loop

└ Show me your GitHub page.

Walk the room looking at browsers, not code. Two minutes per row; note who is behind. While you walk they can pull main and run the feature-1 tests (`uv run pytest tests/test_feature_1.py`) — the command is on the Week 4 page. Don't debug git at the front; point at last week's "when git is confusing" slide.

CLASS 4 · CHECK-IN · 5 MIN

Show me your GitHub page.

- A branch for feature 1.
- A merged pull request.
- Not there yet? Do it first today. Everything else still works.

A chat model writes. An agent **acts**.

A CHAT MODEL

Text in, text out.

It has never seen your folder and cannot run anything. *You* copy code in, *you* paste it out, *you* find out whether it worked.

AN AGENT

The same model, with hands.

It reads your files, runs commands, edits code — and sees the result of each action before choosing the next. It stops when the job is done, or to ask you.

Agent = model + tools + a loop.

2026-09-19

Rules, judge, loop

└ A chat model **writes**. An agent **acts**.

One minute, on their own experience: in Class 2 they asked Codex to show them a line of a data file and it showed the real line — a browser chat could not have. Same model family, different powers. "Agentic" just means: can act, not only answer.

A chat model writes. An agent **acts**.

A CHAT MODEL

Text in, text out.

It has never seen your folder and cannot run anything. You copy code in, you paste it out, you find out whether it worked.

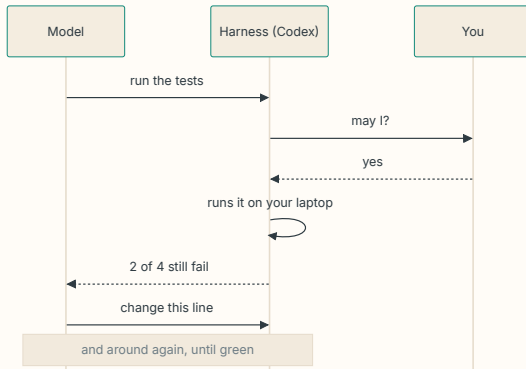
AN AGENT

The same model, with hands.

It reads your files, runs commands, edits code — and sees the result of each action before choosing the next. It stops when the job is done, or to ask you.

Agent = model + tools + a loop.

The model asks. The harness does.



Each arrow from the model is a **tool call** — read a file, edit a file, run a command. That is the whole vocabulary.

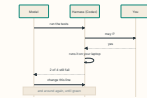
2026-09-19

Rules, judge, loop

└─ The model asks. The harness does.

BEFORE THE STANDARDS · TOOL CALLS

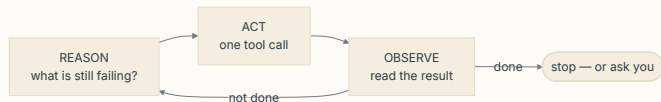
The model asks. The harness does.



Each arrow from the model is a **tool call** — read a file, edit a file, run a command. That is the whole vocabulary.

Narrate the live kick-off later in exactly these words — “that’s a run, that’s a read, that’s an edit” — so the diagram and the demo are the same thing. The approval prompt is where the human sits inside the loop. The sting that sets up standard 2: an agent can run the tests and still **say** they passed without showing you.

The loop has a name: **ReAct**.



THE HARNESS

The model can only *ask*. The software that runs the tool, asks your permission, feeds the result back and decides when to stop is the **harness** — Codex, Claude Code, Cursor.

WHY YOU CARE

Every one of them runs this same loop. Learn the loop once and the tools are interchangeable.

Yao, Zhao, Yu, Du, Shafran, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

2026-09-19

Rules, judge, loop

└ The loop has a name: **ReAct**.

BEFORE THE STANDARDS · REASON, ACT, OBSERVE

The loop has a name: **ReAct**.

THE HARNESS

The model can only *ask*. The software that runs the tool, asks your permission, feeds the result back and decides when to stop is the **harness** — Codex, Claude Code, Cursor.

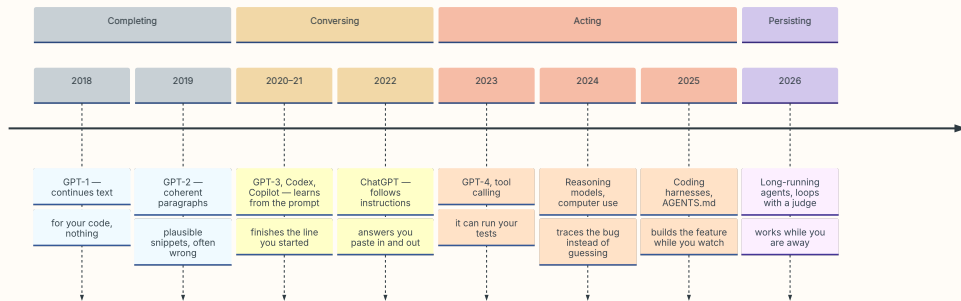
WHY YOU CARE

Every one of them runs this same loop. Learn the loop once and the tools are interchangeable.

Yao, Zhao, Yu, Du, Shafran, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

ReAct = "Reasoning and Acting", Yao et al. 2022 (ICLR 2023, arXiv:2210.03629): a model does far better alternating a thought with an action and reading the result than answering in one go. Every harness since is that loop with better plumbing. The line to land: standards 1-3 are you configuring the harness — the rulebook it loads, the judge it runs, the stop condition of its loop.

Eight years, four kinds of capability.

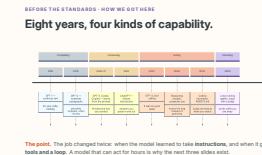


The point. The job changed twice: when the model learned to take **instructions**, and when it got **tools and a loop**. A model that can act for hours is why the next three slides exist.

2026-09-19

Rules, judge, loop

└ Eight years, four kinds of capability.



Two minutes, one sweep left to right. Say the four band names as you go — completing, conversing, acting, persisting — that is the shape to remember, not the model names. Reasoning-row example if you want one: asked why SplitIt's balances were wrong, an older model guesses a plausible edit; a reasoning model works the arithmetic out and lands on the actual line — what a good student does on paper. Sources: GPT-1 2018, GPT-2 2019, GPT-3 2020, Codex 2021, ChatGPT Nov 2022, ReAct 2022, GPT-4 and function calling 2023, o1 and computer use 2024, Claude Code and Codex CLI and agents.md 2025, Codex app 2026. Dates are approximate on purpose.

AGENTS.md — the file it reads first.

WHAT IT IS

A README for agents

Codex reads it at the start of every session, before anything else. So do the other tools. Your repo has had one since Class 2.

WHAT GOES IN

What it cannot guess

The commands. The boundaries. How you want it to report back.

WHAT STAYS OUT

What it can read

Explanations of your own code. Tutorials. "Write clean code." Every extra line hides the ones that matter.

It is code: it lives in git, it gets reviewed, it gets pruned. When the agent misbehaves, the fix is usually one line here — not a longer prompt.

2026-09-19

Rules, judge, loop

AGENTS.md — the file it reads first.

Ask Codex on stage: "What does AGENTS.md tell you to do?" — it recites the file. That is the proof it reads it. The test for every line: would removing it cause a mistake? If not, cut it. Mention the 32 KB cap only if asked; the real cap is attention.

STANDARD 1 • THE RULEBOOK

AGENTS.md — the file it reads first.

WHAT IT IS
A README for agents
Codex reads it at the start of every session, before anything else. So do the other tools. Your repo has had one since Class 2.

WHAT GOES IN
What it cannot guess
The commands. The boundaries. How you want it to report back.

WHAT STAYS OUT
What it can read
Explanations of your own code. Tutorials. "Write clean code." Every extra line hides the ones that matter.

It is code: it lives in git, it gets reviewed, it gets pruned. When the agent misbehaves, the fix is usually one line here — not a longer prompt.

Three rules. One commit.

BOUNDARY

One branch per spec

Never commit straight to `main`.

EVIDENCE

Show me the test output

The output is the proof.
Not a sentence saying so.

SACRED

Never touch the tests

If a test looks wrong, say so and stop.

Add them to `AGENTS.md`, read the diff, commit, push. The exact wording is on the Week 4 page — copy it.

2026-09-19

Rules, judge, loop

└ Three rules. One commit.

Do it on stage: ask Codex to add the three rules under "How to answer" in AGENTS.md, read the diff aloud, accept, then commit in GitHub Desktop. The third one already existed in spirit — now it is explicit, and it is a rule the student enforces. Codex only picks up new rules at the start of a thread, so say "new thread" now. Verbatim wording is on the Week 4 page.

STANDARD 1 • LIVE • 4 MIN

Three rules. One commit.

BOUNDARY One branch per spec Never commit straight to <code>main</code> .	EVIDENCE Show me the test output The output is the proof. Not a sentence saying so.	SACRED Never touch the tests If a test looks wrong, say so and stop.
---	--	--

Add them to `AGENTS.md`, read the diff, commit, push. The exact wording is on the Week 4 page — copy it.

Decide what **done** means. First.

SOMETHING THAT CANNOT LIE

An agent stops when the work *looks* done. Give it a check that answers pass or fail — the failing tests are your to-do list, and green is the finish line.

NEVER MOVE THE GOALPOSTS

An agent that edits a test to make it pass has cheated, not finished.

EVIDENCE, NOT CLAIMS

"All tests pass" is a sentence. The test output is evidence. Ask for it every time.

GREEN IS NOT THE END

The judge only checks what it was told to check. Then you read the diff and click the app. App wrong, tests green? You just found a test that should exist.

2026-09-19

Rules, judge, loop

└─ Decide what **done** means. First.

Run the red tests on stage (the command is on the Week 4 page) and open the test file next to the spec: one test per checklist line. The "green is not the end" card is the one that pays off today — feature 2's form field is not tested at all, so only clicking the app catches a missing one.

STANDARD 2 · THE JUDGE

Decide what **done** means. First.

SOMETHING THAT CANNOT LIE
An agent stops when the work looks done. Give it a check that answers pass or fail — the failing tests are your to-do list, and green is the finish line.

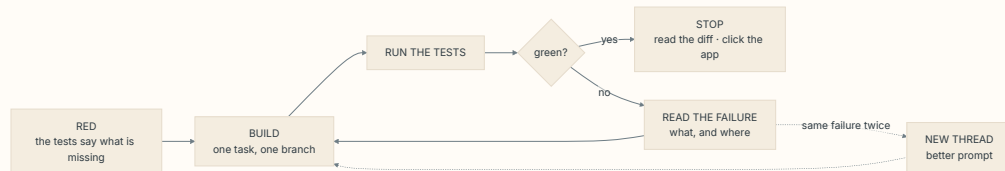
NEVER MOVE THE GOALPOSTS
An agent that edits a test to make it pass has cheated, not finished.

EVIDENCE, NOT CLAIMS
"All tests pass" is a sentence. The test output is evidence. Ask for it every time.

GREEN IS NOT THE END
The judge only checks what it was told to check. Then you read the diff and click the app. App wrong, tests green? You just found a test that should exist.

STANDARD 3 • THE LOOP

Red → build → run → fix → green.



ONE TASK PER LOOP

One bug, or one spec.
Not “fix everything you see”.

LET IT LOOP

It runs the check, reads
the failure, tries again.
You read the result — you
are not the messenger.

KNOW WHEN TO STOP

Green, or twice round on
the same failure. Then:
new thread, better
prompt.

2026-09-19

Rules, judge, loop

Red → build → run → fix → green.



The three parts are the whole lesson: a rulebook, a judge, a loop. Everything else in agentic coding is a variation. Today the loop runs inside one Codex turn and you watch it; the same idea one level up is a script that re-feeds the prompt until the judge passes — that is Part II, with the team project.

Every good prompt has four parts.

1 • THE TASK

One thing.

Name the single spec or bug you want built. Nothing else.

2 • THE BOUNDARY

Where it may go.

This branch. And never the tests.

3 • THE JUDGE

How it knows it is done.

Run the tests after every change, and keep going until they pass.

4 • THE EVIDENCE

What to show you.

The final test output, and every file it touched.

The prompt written out in full is on the Week 4 page. Copy it and change two names.

2026-09-19

Rules, judge, loop

└ Every good prompt has four parts.

Four parts, in every loop prompt they will ever write. Get them to name the four parts back to you before you paste it. New thread before pasting — the AGENTS.md rules from five minutes ago are only read at the start of a thread.

STANDARD 3 • THE SHAPE OF THE ASK

Every good prompt has four parts.

1 • THE TASK

One thing.

Name the single spec or bug you want built. Nothing else.

2 • THE BOUNDARY

Where it may go.

This branch. And never the tests.

3 • THE JUDGE

How it knows it is done.

Run the tests after every change, and keep going until they pass.

4 • THE EVIDENCE

What to show you.

The final test output, and every file it touched.

The prompt written out in full is on the Week 4 page. Copy it and change two names.

Watch it work. Then check its work.

- 1 Branch. Name it after the spec.
- 2 Red first. See exactly what is missing before anything is written.
- 3 New thread. Paste the prompt. Then sit on your hands.
- 4 Watch it loop. Edit, run, read the failure, edit again.
- 5 Green. Now don't trust it. Read the diff: did it change only what the task needed?
- 6 Click the app. The part no test covers is the part only you can catch.
- 7 Commit, push, pull request, merge.

2026-09-19

Rules, judge, loop

└─ Watch it work. Then check its work.

Let it loop for real; 2-4 rounds is normal. Narrate it as tool calls: "that's a run — two still red — it's reading the failure — that's an edit". Commands and file names are on the Week 4 page if you need them on screen. The two things it commonly misses on this feature: it forgets to refresh the data files from the seed (so the tests stay green while the app crashes on load) and it never adds the form field (nothing tests that screen). Both are the "green is not the end" lesson — do not fix them silently, show them. If it edits a test: stop, discard, point at the rule, re-prompt.

CLASS 4 · LIVE · FEATURE 2 · 13 MIN

Watch it work. Then check its work.

- 1 Branch. Name it after the spec.
- 2 Red first. See exactly what is missing before anything is written.
- 3 New thread. Paste the prompt. Then sit on your hands.
- 4 Watch it loop. Edit, run, read the failure, edit again.
- 5 Green. Now don't trust it. Read the diff: did it change only what the task needed?
- 6 Click the app. The part no test covers is the part only you can catch.
- 7 Commit, push, pull request, merge.

One bug on `main`. One feature on a branch.

SPLITIT

Bug 2, then feature 2

Removing a member crashes the page.
Then expense categories.

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then
conversion by source.



Files, test names and the prompt: Week 4 page.

2026-09-19

Rules, judge, loop

└ One bug on `main`. One feature on a branch.



Roam for two things: did they ask for the test output, and did they read the diff before accepting. Never answer a failing test — "what does the last line say?". SplitIt bug 002: the test demands a helper that also removes the member's expenses — the assertion message says so. Tiny CRM bug 003 has a twist: stamping the close date is not enough, the month counter gives up on the whole list when any won lead has no date, and the seed has five of those; the fix is to skip them, not to return zero. Anyone still on feature 1: that first, then the bug; feature 2 becomes homework.

Five things you will see.

- 1 It edited a test. Discard it. Say the rule out loud. Run again.
- 2 "Done" — but the tests are red. Ask for the output. The output is the judge.
- 3 The same failure, three times. Stop. Read it yourself. New thread.
- 4 It fixed three other things too. Discard them. One task per loop.
- 5 Green, but the app is wrong. The judge missed something. Write down what.

Every line here is a candidate rule for your `AGENTS.md`. Keep the list.

2026-09-19

Rules, judge, loop

└ Five things you will see.

Five things you will see.

- 1 It edited a test. Discard it. Say the rule out loud. Run again.
- 2 "Done" — but the tests are red. Ask for the output. The output is the judge.
- 3 The same failure, three times. Stop. Read it yourself. New thread.
- 4 It fixed three other things too. Discard them. One task per loop.
- 5 Green, but the app is wrong. The judge missed something. Write down what.

Every line here is a candidate rule for your `AGENTS.md`. Keep the list.

What did it get wrong? Write the rule.

- The worst thing the agent did today. One sentence each.
- Turn it into a rule: short, specific, and you could tell whether it was followed.
- Add it to `AGENTS.md`. Say why in the commit message. Push.
- Three groups read theirs out.

That is how a rulebook grows — from what went wrong, one line at a time. Not from a longer prompt.

2026-09-19

Rules, judge, loop

└─What did it get wrong? Write the rule.

Collect the rules on the board; the good ones become the course's AGENTS.md for Block 2. Expect: "refresh the data after changing the seed files", "don't edit the live data by hand", "show me the diff before committing", "one file at a time". Reject vague ones ("be careful") — ask "how would you know it followed this?".

What did it get wrong? Write the rule.

- The worst thing the agent did today. One sentence each.
- Turn it into a rule: short, specific, and you could tell whether it was followed.
- Add it to `AGENTS.md`. Say why in the commit message. Push.
- Three groups read theirs out.

That is how a rulebook grows — from what went wrong, one line at a time. Not from a longer prompt.

Next week you build alone.

- Finish feature 2. Green, merged, pushed.
- Your rule is in `AGENTS.md` with the reason in the commit message.
- Push before you close the laptop.

Rulebook · judge · loop.

Block 2 is those three, on a spec of your own.

2026-09-19

Rules, judge, loop

└─ Next week you build alone.

Nothing to hand in. Next week's check-in: feature 2 merged, and a commit on AGENTS.md. Part II of the standards — walking away: scripted loops, goals, a second agent reviewing — comes with the team project in Block 3.

CLASS 4 · BEFORE NEXT WEEK

Next week you build alone.

- Finish feature 2. Green, merged, pushed.
- Your rule is in `AGENTS.md`, with the reason in the commit message.
- Push before you close the laptop.

Rulebook · judge · loop.
Block 2 is those three, on a spec of your own.